Advanced Topics

This guide covers advanced ReportViewerCore features and optimization techniques.

Table of Contents

Subreports

Implementing Subreports

Subreports allow you to embed one report inside another.

```
public class ReportWithSubreports
{
    public byte[] GenerateReport()
    {
        using var report = new LocalReport();

        // Load main report
        using var stream = File.OpenRead("Reports/MainReport.rdlc");
        report.LoadReportDefinition(stream);

        // Add main data source
        report.DataSources.Add(new ReportDataSource("MainData",
GetMainData()));

        // Handle subreport processing
        report.SubreportProcessing += OnSubreportProcessing;

        return report.Render("PDF");
    }

    private void OnSubreportProcessing(object sender,
SubreportProcessingEventArgs e)
    {
        // e.ReportPath contains the subreport name/path
        // e.DataSourceNames contains the expected data source names

        switch (e.ReportPath)
```

```
        {
            case "SubReport1":
                // Get parent report parameters
                var parentId = e.Parameters["ParentId"].Values[0];
                var data = GetSubReport1Data(int.Parse(parentId));
                e.DataSources.Add(new ReportDataSource("SubData", data));
                break;

            case "SubReport2":
                var category = e.Parameters["Category"].Values[0];
                var data2 = GetSubReport2Data(category);
                e.DataSources.Add(new ReportDataSource("CategoryData", data2));
                break;
        }
    }
}
```

Passing Parameters to Subreports

```
// In your main report, you can pass parameters to subreports
// The parameter values come from fields in the parent dataset

// In the subreport event handler:
private void OnSubreportProcessing(object sender, SubreportProcessingEventArgs
e)
{
    // Access parameters passed from parent report
    foreach (var param in e.Parameters)
    {
        Console.WriteLine($"{param.Name}: {string.Join(", ", param.Values)}");
    }

    // Add data sources based on parameters
    var orderId = e.Parameters["OrderId"].Values[0];
    var orderDetails = GetOrderDetails(int.Parse(orderId));
    e.DataSources.Add(new ReportDataSource("OrderDetails", orderDetails));
}
```

Custom Code and Assemblies

Using Custom Code in Reports

You can use custom .NET code in your reports for calculations and formatting.

In Report Designer

1. Open Report Properties
2. Go to Code tab

3. Add custom functions:

```
Public Function GetDiscount(ByVal quantity As Integer) As Decimal
    If quantity >= 100 Then
        Return 0.15D
    ElseIf quantity >= 50 Then
        Return 0.10D
    ElseIf quantity >= 10 Then
        Return 0.05D
    Else
        Return 0D
    End If
End Function

Public Function FormatStatus(ByVal status As String) As String
    Select Case status.ToUpper()
        Case "ACTIVE"
            Return "✓ Active"
        Case "PENDING"
            Return "⏳ Pending"
        Case "CANCELLED"
            Return "✗ Cancelled"
        Case Else
            Return status
    End Select
End Function
```

Use in Report Expression

```
=Code.GetDiscount(Fields!Quantity.Value)
=Code.FormatStatus(Fields!Status.Value)
```

Using External Assemblies

```
// 1. Create a custom assembly
namespace MyCompany.Reporting
{
    public class ReportFunctions
    {
        public static decimal CalculateTax(decimal amount, string region)
        {
            return region switch
            {
                "CA" => amount * 0.0725m,
                "NY" => amount * 0.08m,
                "TX" => amount * 0.0625m,
```

```
                _ => amount * 0.05m
            };
        }

        public static string FormatCurrency(decimal amount, string
currencyCode)
        {
            return currencyCode switch
            {
                "USD" => $"${amount:N2}",
                "EUR" => $"€{amount:N2}",
                "GBP" => $"£{amount:N2}",
                _ => amount.ToString("N2")
            };
        }
    }
}
```

Configure Report to Use Assembly

In RDLC designer:

1. Report Properties > References
2. Add reference to your assembly DLL
3. Add class instance (optional)

Use in expressions:

```
=MyCompany.Reporting.ReportFunctions.CalculateTax(Fields!Amount.Value,
Fields!Region.Value)
```

Server Reports (SSRS)

Connecting to Reporting Services

```
public class ServerReportExample
{
    public byte[] GenerateServerReport(string reportPath)
    {
        var report = new ServerReport();

        // Configure server connection
        report.ReportServerUrl = new Uri("http://localhost/ReportServer");
        report.ReportPath = reportPath; // e.g., "/Sales/InvoiceReport"

        // Set credentials
        report.ReportServerCredentials = new ReportServerCredentials
```

```
        {
            NetworkCredentials = new NetworkCredential("username", "password",
"DOMAIN")
        };

        // Set parameters
        report.SetParameters(new[] {
            new ReportParameter("StartDate", "2024-01-01"),
            new ReportParameter("EndDate", "2024-12-31")
        });

        // Render
        return report.Render("PDF");
    }
}

public class ReportServerCredentials : IReportServerCredentials
{
    public NetworkCredential NetworkCredentials { get; set; }

    public bool GetFormsCredentials(out Cookie authCookie, out string userName,
        out string password, out string authority)
    {
        authCookie = null;
        userName = null;
        password = null;
        authority = null;
        return false;
    }

    public WindowsIdentity ImpersonationUser => null;
}
```

Bearer Token Authentication

```
public class BearerTokenReportServer
{
    public byte[] GenerateReport(string reportPath, string bearerToken)
    {
        var report = new ServerReport();

        report.ReportServerUrl = new Uri("https://powerbi.com/reportserver");
        report.ReportPath = reportPath;

        // Use bearer token
        report.BearerToken = bearerToken;

        return report.Render("PDF");
    }
}
```

Performance Optimization

Report Caching

```
public class CachedReportService
{
    private readonly IMemoryCache _cache;
    private readonly IWebHostEnvironment _env;

    public CachedReportService(IMemoryCache cache, IWebHostEnvironment env)
    {
        _cache = cache;
        _env = env;
    }

    public byte[] GenerateReport(string reportName, object data, string format)
    {
        // Cache the report definition
        var definitionKey = $"report-definition-{reportName}";

        if (!_cache.TryGetValue(definitionKey, out byte[] reportDefinition))
        {
            var path = Path.Combine(_env.ContentRootPath, "Reports", $"
{reportName}.rdlc");
            reportDefinition = File.ReadAllBytes(path);

            _cache.Set(definitionKey, reportDefinition, new
MemoryCacheEntryOptions
            {
                AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24),
                Size = reportDefinition.Length
            });
        }

        // Generate report
        using var report = new LocalReport();
        using var stream = new MemoryStream(reportDefinition);
        report.LoadReportDefinition(stream);
        report.DataSources.Add(new ReportDataSource("Data", data));

        return report.Render(format);
    }
}
```

Compiled Report Caching

```
public class CompiledReportCache
{
    private readonly ConcurrentDictionary<string, LocalReport> _compiledReports
= new();

    public LocalReport GetOrCompileReport(string reportPath)
    {
        return _compiledReports.GetOrAdd(reportPath, path =>
        {
            var report = new LocalReport();
            using var stream = File.OpenRead(path);
            report.LoadReportDefinition(stream);
            return report;
        });
    }

    public byte[] GenerateReport(string reportName, object data, string format)
    {
        // Note: This approach requires careful memory management
        // Consider using a factory pattern instead for production use

        var template = GetOrCompileReport($"Reports/{reportName}.rdlc");

        // Create a new instance for this request
        using var report = CloneReport(template);
        report.DataSources.Add(new ReportDataSource("Data", data));

        return report.Render(format);
    }
}
```

Streaming Large Reports

```
public class StreamingReportService
{
    public Stream GenerateReportStream(string reportName, object data, string
format)
    {
        var stream = new MemoryStream();

        using var report = new LocalReport();
        LoadReport(report, reportName, data);

        // Use streaming render
        report.Render(format, null, (name, extension, encoding, mimeType,
willSeek) =>
        {
            return stream;
        }, out Warning[] warnings);
```

```
        stream.Position = 0;
        return stream;
    }

    public async Task<IActionResult> StreamReportToResponse(
        HttpContext context,
        string reportName,
        object data)
    {
        context.Response.ContentType = "application/pdf";
        context.Response.Headers.Add("Content-Disposition",
            $"attachment; filename={reportName}.pdf");

        using var report = new LocalReport();
        LoadReport(report, reportName, data);

        // Stream directly to response
        report.Render("PDF", null, (name, extension, encoding, mimeType,
willSeek) =>
        {
            return context.Response.Body;
        }, out Warning[] warnings);

        return new EmptyResult();
    }
}
```

Parallel Report Generation

```
public class ParallelReportService
{
    public async Task<Dictionary<string, byte[]>> GenerateMultipleReportsAsync(
        IEnumerable<ReportRequest> requests)
    {
        var tasks = requests.Select(async request =>
        {
            var report = await Task.Run(() => GenerateReport(request));
            return new { request.ReportName, Report = report };
        });

        var results = await Task.WhenAll(tasks);

        return results.ToDictionary(r => r.ReportName, r => r.Report);
    }

    private byte[] GenerateReport(ReportRequest request)
    {
        using var report = new LocalReport();
        // ... generate report
```

```
        return report.Render(request.Format);
    }
}
```

Custom Device Info

Customize rendering behavior with device info XML:

```
public class CustomDeviceInfo
{
    public byte[] GeneratePdfWithOptions()
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Custom PDF settings
        var deviceInfo = @"
            <DeviceInfo>
                <OutputFormat>PDF</OutputFormat>
                <PageWidth>8.5in</PageWidth>
                <PageHeight>11in</PageHeight>
                <MarginTop>0.5in</MarginTop>
                <MarginLeft>0.5in</MarginLeft>
                <MarginRight>0.5in</MarginRight>
                <MarginBottom>0.5in</MarginBottom>
                <EmbedFonts>True</EmbedFonts>
                <HumanReadablePDF>True</HumanReadablePDF>
            </DeviceInfo>";

        return report.Render("PDF", deviceInfo);
    }

    public byte[] GenerateHtmlWithOptions()
    {
        using var report = new LocalReport();
        LoadReport(report);

        var deviceInfo = @"
            <DeviceInfo>
                <Toolbar>False</Toolbar>
                <StyleStream>True</StyleStream>
                <StreamRoot>/Reports/</StreamRoot>
                <LinkTarget>_blank</LinkTarget>
            </DeviceInfo>";

        return report.Render("HTML5", deviceInfo);
    }

    public byte[] GenerateExcelWithOptions()
    {
```

```
        using var report = new LocalReport();
        LoadReport(report);

        var deviceInfo = @"
            <DeviceInfo>
                <SimplePageHeaders>True</SimplePageHeaders>
                <OmitDocumentMap>False</OmitDocumentMap>
            </DeviceInfo>";

        return report.Render("EXCELOPENXML", deviceInfo);
    }
}
```

Report Warnings and Debugging

Handling Warnings

```
public class ReportWithWarnings
{
    public byte[] GenerateReportWithWarnings(out List<string> warnings)
    {
        using var report = new LocalReport();
        LoadReport(report);

        Warning[] renderWarnings;
        var result = report.Render("PDF", null, PageCountMode.Actual,
            out string mimeType,
            out string encoding,
            out string fileExtension,
            out string[] streams,
            out renderWarnings);

        // Process warnings
        warnings = renderWarnings
            .Select(w => $"[{w.Severity}] {w.Code}: {w.Message} (Object:
{w.ObjectName})")
            .ToList();

        // Log warnings
        foreach (var warning in warnings)
        {
            Console.WriteLine(warning);
        }

        return result;
    }
}
```

Debug Mode

```
public class DebugReportService
{
    private readonly ILogger<DebugReportService> _logger;

    public byte[] GenerateReportWithDebugInfo(string reportName, object data)
    {
        var sw = Stopwatch.StartNew();

        using var report = new LocalReport();

        _logger.LogInformation("Loading report: {ReportName}", reportName);
        LoadReport(report, reportName);

        _logger.LogInformation("Adding data sources");
        report.DataSources.Add(new ReportDataSource("Data", data));

        _logger.LogInformation("Rendering report");
        var result = report.Render("PDF", null, PageCountMode.Actual,
            out _,
            out _,
            out _,
            out _,
            out Warning[] warnings);

        sw.Stop();

        _logger.LogInformation(
            "Report generated in {ElapsedMs}ms with {WarningCount} warnings",
            sw.ElapsedMilliseconds,
            warnings.Length);

        foreach (var warning in warnings)
        {
            _logger.LogWarning(
                "Report warning: {Code} - {Message}",
                warning.Code,
                warning.Message);
        }

        return result;
    }
}
```

External Images

Enabling External Images

```
public class ReportWithExternalImages
{
```

```
    public byte[] GenerateReport()
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Enable external images (security consideration!)
        report.EnableExternalImages = true;

        // Images in report can reference external URLs:
        // ="http://example.com/images/logo.png"

        return report.Render("PDF");
    }
}
```

Loading Images from Database

```
public class ReportWithDatabaseImages
{
    public byte[] GenerateReportWithImages(int productId)
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Get product data with images
        var product = GetProductWithImage(productId);

        // Add data source with image byte array
        var dataSource = new[]
        {
            new
            {
                product.Name,
                product.Description,
                // Image must be byte array
                ProductImage = product.ImageBytes
            }
        };

        report.DataSources.Add(new ReportDataSource("Product", dataSource));

        // In RDLC, use expression:
        // =Fields!ProductImage.Value
        // With image Source property set to "Database"

        return report.Render("PDF");
    }
}
```

Dynamic Data Sources

Multiple Data Sources

```
public class MultiDataSourceReport
{
    public byte[] GenerateComplexReport()
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Add multiple data sources
        report.DataSources.Add(new ReportDataSource("Customers",
GetCustomers()));
        report.DataSources.Add(new ReportDataSource("Orders", GetOrders()));
        report.DataSources.Add(new ReportDataSource("Products",
GetProducts()));
        report.DataSources.Add(new ReportDataSource("OrderDetails",
GetOrderDetails()));

        return report.Render("PDF");
    }
}
```

Dynamic Data Source Selection

```
public class DynamicDataSourceReport
{
    public byte[] GenerateReport(string dataSourceType)
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Dynamically choose data source
        IEnumerable data = dataSourceType switch
        {
            "SQL" => GetDataFromSql(),
            "API" => GetDataFromApi(),
            "File" => GetDataFromFile(),
            "Cache" => GetDataFromCache(),
            _ => throw new ArgumentException("Invalid data source type")
        };

        report.DataSources.Add(new ReportDataSource("Data", data));

        return report.Render("PDF");
    }
}
```

Filtering Data Before Rendering

```
public class FilteredReport
{
    public byte[] GenerateFilteredReport(FilterCriteria criteria)
    {
        using var report = new LocalReport();
        LoadReport(report);

        // Get and filter data
        var allData = GetAllData();
        var filteredData = allData
            .Where(d => d.Date >= criteria.StartDate && d.Date <=
criteria.EndDate)
            .Where(d => criteria.Categories.Contains(d.Category))
            .Where(d => d.Amount >= criteria.MinAmount)
            .OrderBy(d => d.Date)
            .ToList();

        report.DataSources.Add(new ReportDataSource("Data", filteredData));

        // Add filter info as parameters
        report.SetParameters(new[] {
            new ReportParameter("FilterApplied", "Yes"),
            new ReportParameter("RecordCount", filteredData.Count.ToString())
        });

        return report.Render("PDF");
    }
}
```

Next Steps

- Troubleshooting - Common issues and solutions
- Review the official RDLC documentation for report design
- Explore the sample projects in this repository